EECE5644 Fall 2026 – Assignment 2

David Abu

All experiments were implemented in Python 3 (NumPy 2.5, SciPy 1.18, Matplotlib 3.11) with fixed random seeds, so every number below is reproducible from the code in the Appendix.

Question 1

Data

Each sample's label is drawn with $P(L{=}1) = 0.4$; given $L = l$, one of the two equally weighted Gaussian components $g(\mathbf{x}|\mathbf{m}_{l1}, C)$, $g(\mathbf{x}|\mathbf{m}_{l2}, C)$ is chosen and $\mathbf{x}$ is drawn from it, with $C = \mathrm{diag}(0.75, 1.25)$. Datasets $D^{20}_{train}, D^{200}_{train}, D^{2000}_{train}, D^{10K}_{validate}$ were generated independently. The validation set contains $N_0 = 5949$, $N_1 = 4051$ samples.

Part 1: Theoretically optimal (min-P(error)) classifier

With 0–1 loss, the minimum-risk rule is the MAP rule: decide $D = 1$ iff $P(L{=}1|\mathbf{x}) > P(L{=}0|\mathbf{x})$, i.e. the likelihood-ratio test

$$\frac{p(\mathbf{x}|L{=}1)}{p(\mathbf{x}|L{=}0)} = \frac{\frac{1}{2}g(\mathbf{x}|\mathbf{m}_{11}, C) + \frac{1}{2}g(\mathbf{x}|\mathbf{m}_{12}, C)}{\frac{1}{2}g(\mathbf{x}|\mathbf{m}_{01}, C) + \frac{1}{2}g(\mathbf{x}|\mathbf{m}_{02}, C)} \underset{D=0}{\overset{D=1}{\gtrless}} \gamma^* = \frac{P(L{=}0)}{P(L{=}1)} = \frac{0.6}{0.4} = 1.5.$$

The discriminant score used for the ROC is the log-likelihood ratio $s(\mathbf{x}) = \ln p(\mathbf{x}|L{=}1) - \ln p(\mathbf{x}|L{=}0)$, thresholded at $\ln\gamma$. Sweeping $\ln\gamma$ over all observed score values gives the ROC curve ($P_{FA} = P(D{=}1|L{=}0)$ vs. $P_D = P(D{=}1|L{=}1)$); for each threshold the empirical error is

$$P(\text{error}) = P_{FA}\, P(L{=}0) + (1 - P_D)\, P(L{=}1)$$

with priors estimated from the validation labels.

	$L = 0$	$L = 1$
$D = 0$	5011	1855
$D = 1$	938	2196

Table 1: Decision/label counts of the optimal classifier ($\gamma^* = 1.5$) on $D^{10K}_{validate}$.

Estimated min-P(error) $= \dfrac{938 + 1855}{10000} = \mathbf{0.2793}$ ($P_{FA} = 0.1577$, $P_D = 0.5421$). The empirically best threshold on the ROC is $\ln\gamma = 0.433$ (vs. theoretical 0.405) with $P(\text{error}) = 0.2779$; the small gap is finite-sample noise, confirming the theory.

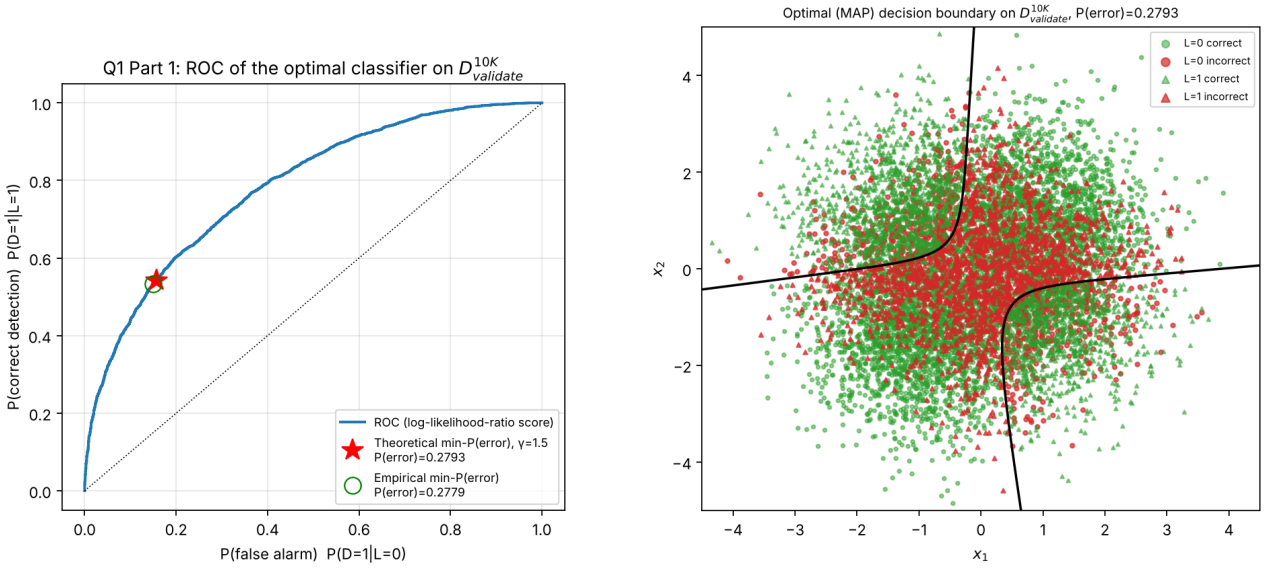


Figure 1: Left: ROC of the log-likelihood-ratio score with the theoretical min-P(error) operating point (red star) and the empirical optimum (green circle). Right: optimal decision boundary on $D_{validate}^{10K}$ (green = correct, red = misclassified).

The boundary is a pair of hyperbola-like curves: class 0 occupies the 3rd and 1st quadrants and class 1 the 2nd and 4th (an XOR-like arrangement), so no single line can separate them.

Part 2: Logistic approximations trained by maximum likelihood

Let $h(\mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^T \mathbf{z}(\mathbf{x}))$, $\sigma(a) = 1/(1+e^{-a})$, model $P(L{=}1|\mathbf{x})$, with $\mathbf{z}(\mathbf{x}) = [1, x_1, x_2]^T$ (linear) or $\mathbf{z}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]^T$ (quadratic). For iid training data $\{(\mathbf{x}_n, l_n)\}_{n=1}^N$ the likelihood is $\prod_n h_n^{l_n}(1-h_n)^{1-l_n}$, so the ML problem is

$$\hat{\mathbf{w}}_{ML} = \arg\min_{\mathbf{w}} \; -\frac{1}{N}\sum_{n=1}^{N}\Big[l_n \ln h(\mathbf{x}_n,\mathbf{w}) + (1-l_n)\ln\big(1-h(\mathbf{x}_n,\mathbf{w})\big)\Big], \qquad \nabla_{\mathbf{w}} = \frac{1}{N}\sum_n \big(h(\mathbf{x}_n,\mathbf{w})-l_n\big)\mathbf{z}(\mathbf{x}_n).$$

The objective is convex in $\mathbf{w}$; it was minimized with BFGS (`scipy.optimize.minimize`) using the analytic gradient, starting from $\mathbf{w} = \mathbf{0}$ (implemented stably as $\ln(1+e^a) - la$, $a = \mathbf{w}^T\mathbf{z}$).

Classification rule. Since $h(\mathbf{x}, \hat{\mathbf{w}}) \approx P(L{=}1|\mathbf{x})$, the min-P(error) (MAP) rule becomes: decide $D = 1$ iff $h(\mathbf{x}, \hat{\mathbf{w}}) \geq 0.5$, equivalently $\hat{\mathbf{w}}^T\mathbf{z}(\mathbf{x}) \geq 0$. (The priors are already absorbed into the learned bias term, because the training labels are sampled with the true priors.)

Model	N_{train}	train P(error)	**validation P(error)**	$\hat{\mathbf{w}}_{ML}$
logistic-linear	20	0.250	**0.4198**	$[-1.145, 0.357, 0.263]$
logistic-linear	200	0.410	**0.3793**	$[-0.286, -0.088, 0.232]$
logistic-linear	2000	0.422	**0.4074**	$[-0.337, 0.013, 0.110]$
logistic-quadratic	20	0.150	**0.3410**	$[-8.38, 19.06, 16.22, -11.85, -23.58, -5.67]$
logistic-quadratic	200	0.330	**0.2851**	$[-0.335, -0.018, 0.183, 0.067, -0.511, -0.010]$
logistic-quadratic	2000	0.279	**0.2795**	$[-0.368, 0.000, 0.119, 0.065, -0.869, -0.050]$
Optimal (Part 1)	–	–	**0.2793**	–

Table 2: P(error) estimated by counting decisions on $D^{10K}_{validate}$. For reference, always deciding $D = 0$ gives ≈ 0.405.

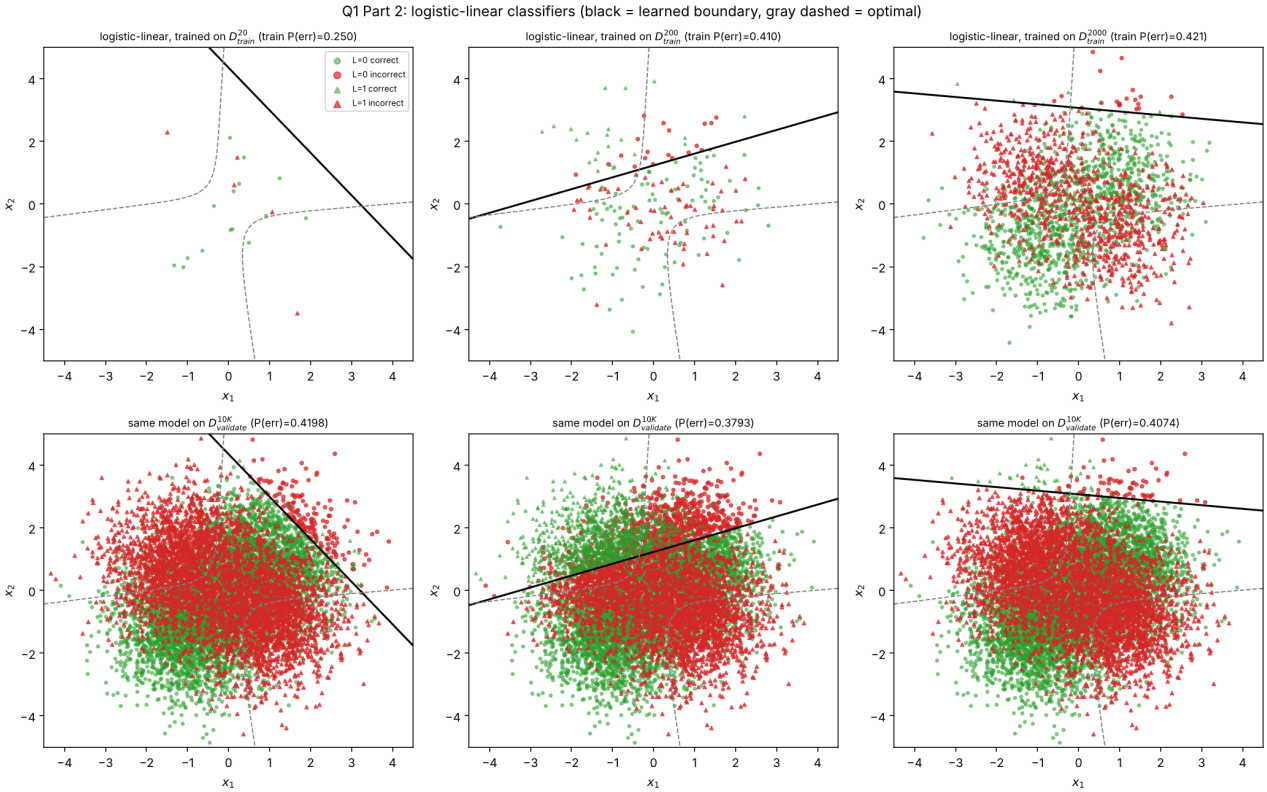


Figure 2: Logistic-linear classifiers. Top: training sets; bottom: validation set. Black = learned boundary, gray dashed = optimal boundary.

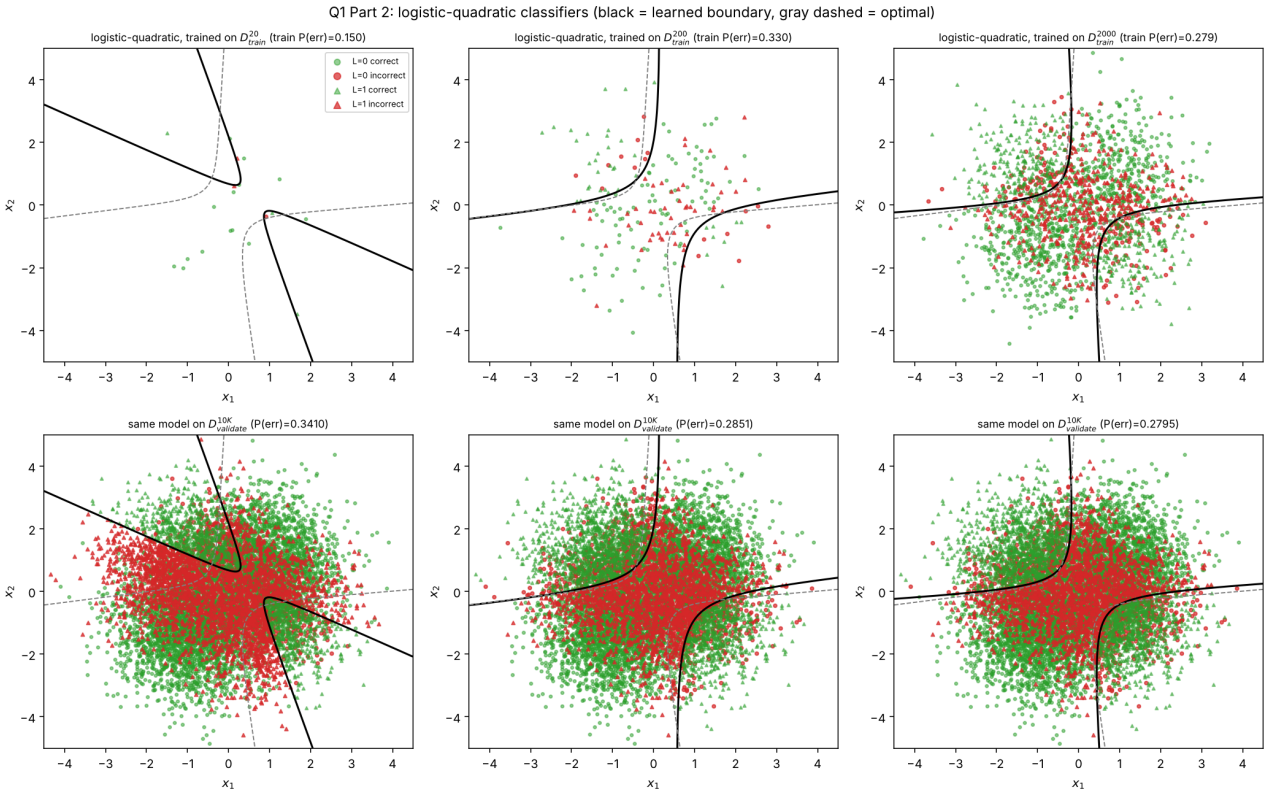


Figure 3: Logistic-quadratic classifiers, same layout.

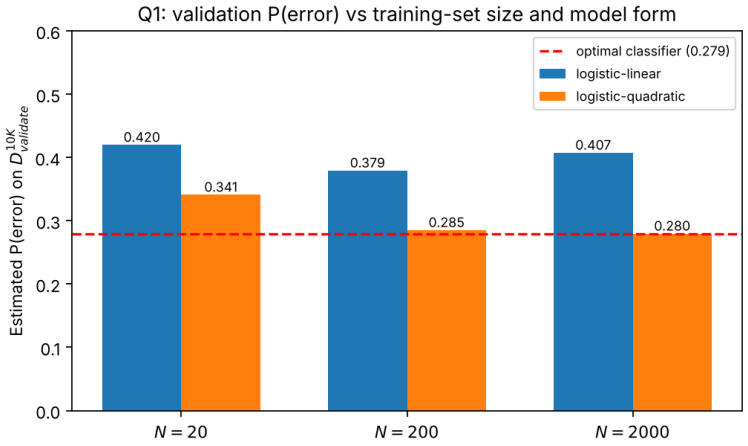


Figure 4: Validation P(error) vs. training size and model form; red dashed = optimal classifier.

Discussion

- **Function form dominates.** The Bayes boundary is XOR-like (opposite quadrants share a class), so a linear boundary cannot represent it. Even with 2000 samples the logistic-linear model reaches only 0.41, essentially no better than always predicting the majority class (0.405); its weights on x_1, x_2 are near zero because the classes have nearly identical marginal means.

More data cannot fix this bias.

- **The quadratic model contains the right structure.** The cross term x_1x_2 (weight ≈ -0.87 at N=2000) is negative in the class-0 quadrants and positive in the class-1 quadrants, which is exactly the XOR pattern. With $N = 2000$ it attains 0.2795, statistically indistinguishable from the optimal 0.2793, and its boundary overlays the optimal one (Fig. above). The true posterior is not exactly logistic-quadratic (it is a ratio of Gaussian mixtures), but a quadratic log-odds is an excellent approximation where the data lie.
- **Sample size controls variance/overfitting.** With $N = 20$ the quadratic model nearly separates the training data (train error 0.15) and its weights blow up ($|w| \sim 20$), giving a distorted boundary and validation error 0.34. With $N = 200$ the estimate stabilizes (0.285) and with $N = 2000$ it converges to the Bayes boundary. The linear $N = 20$ model also shows the optimism of training error (0.25 train vs. 0.42 validation).
- **Relation to Part 1.** The optimal classifier is a lower bound on achievable P(error); every trained classifier is at or above it, and only the correctly specified-enough model with enough data approaches it.

Question 2

Model and derivations

Let $\mathbf{z}(\mathbf{x}) \in \mathbb{R}^{10}$ collect all monomials of $\mathbf{x} = [x_1, x_2]^T$ of total degree ≤ 3:

$$\mathbf{z}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3]^T,$$

so $c(\mathbf{x}, \mathbf{w}) = \mathbf{w}^T\mathbf{z}(\mathbf{x})$ and $y_n = \mathbf{w}^T\mathbf{z}_n + v_n$, $v_n \sim \mathcal{N}(0, \sigma^2)$ iid. Stack $Z = [\mathbf{z}_1, \ldots, \mathbf{z}_N]^T \in \mathbb{R}^{N\times 10}$, $\mathbf{y} = [y_1, \ldots, y_N]^T$.

ML. $p(D|\mathbf{w}) = \prod_n (2\pi\sigma^2)^{-1/2} \exp\big(-\frac{(y_n - \mathbf{w}^T\mathbf{z}_n)^2}{2\sigma^2}\big)$, so

$$\ln p(D|\mathbf{w}) = -\frac{N}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|\mathbf{y} - Z\mathbf{w}\|^2 \;\Rightarrow\; \hat{\mathbf{w}}_{ML} = \arg\min_{\mathbf{w}} \|\mathbf{y} - Z\mathbf{w}\|^2.$$

Setting $\nabla_{\mathbf{w}} = -2Z^T(\mathbf{y} - Z\mathbf{w}) = 0$:

$$\boxed{\hat{\mathbf{w}}_{ML} = (Z^TZ)^{-1}Z^T\mathbf{y} = \Big(\sum_n \mathbf{z}_n\mathbf{z}_n^T\Big)^{-1}\sum_n \mathbf{z}_n y_n.}$$

MAP. With prior $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \gamma I)$, $\ln p(\mathbf{w}) = -\frac{1}{2\gamma}\|\mathbf{w}\|^2 + \text{const}$, and

$$\hat{\mathbf{w}}_{MAP} = \arg\max_{\mathbf{w}} \big[\ln p(D|\mathbf{w}) + \ln p(\mathbf{w})\big] = \arg\min_{\mathbf{w}} \Big[\frac{1}{\sigma^2}\|\mathbf{y} - Z\mathbf{w}\|^2 + \frac{1}{\gamma}\|\mathbf{w}\|^2\Big].$$

Setting the gradient $-\frac{2}{\sigma^2}Z^T(\mathbf{y} - Z\mathbf{w}) + \frac{2}{\gamma}\mathbf{w}$ to zero:

$$\boxed{\hat{\mathbf{w}}_{MAP} = \Big(Z^TZ + \frac{\sigma^2}{\gamma}I\Big)^{-1}Z^T\mathbf{y} = \Big(\sum_n \mathbf{z}_n\mathbf{z}_n^T + \frac{\sigma^2}{\gamma}I\Big)^{-1}\sum_n \mathbf{z}_n y_n.}$$

This is ridge regression with $\lambda = \sigma^2/\gamma$. **Relation to ML:** as $\gamma \to \infty$ (uninformative prior) $\lambda \to 0$ and $\hat{\mathbf{w}}_{MAP} \to \hat{\mathbf{w}}_{ML}$; as $\gamma \to 0$ the prior dominates and $\hat{\mathbf{w}}_{MAP} \to \mathbf{0}$. For finite γ the MAP estimate is the ML estimate shrunk toward zero: in the eigenbasis of $Z^T Z$ (eigenvalues μ_j) each component is scaled by $\mu_j/(\mu_j + \lambda)$.

Since σ^2 is not given, it was estimated from the ML residuals on the training set, $\hat{\sigma}^2 = \|\mathbf{y} - Z\hat{\mathbf{w}}_{ML}\|^2/(N - 10) = 3.636$. Only the ratio $\lambda = \sigma^2/\gamma$ matters, so a different $\hat{\sigma}^2$ simply shifts the γ axis.

Experiments

Data were produced by the course-provided `hw2q2.py` generator ($N_{train} = 100$, $N_{validate} = 1000$; generator logic copied unchanged, with plotting removed and a fixed seed). γ was swept over 161 log-spaced values in $[10^{-10}, 10^{6}]$ ($m = 10$, $n = 6$); every model was evaluated by the average squared error on the validation set, $\frac{1}{N_v}\sum_n (y_n - \hat{\mathbf{w}}^T \mathbf{z}(\mathbf{x}_n))^2$.

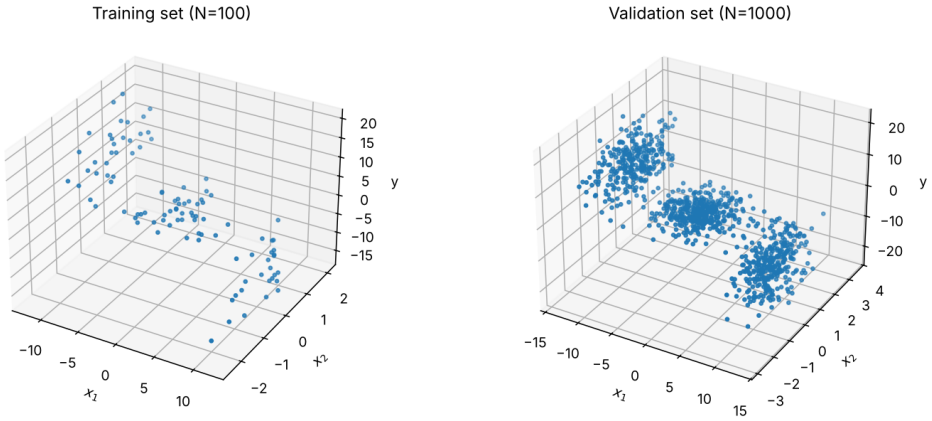


Figure 5: Training and validation data from the provided generator.

ML: training MSE = 3.272, **validation MSE** = 5.455.
$\hat{\mathbf{w}}_{ML} = [0.128, 0.172, 0.167, -0.0024, -0.0205, 0.0733, -0.0108, 0.0110, -0.0247, -0.470]$.

γ	validation MSE	$\|\hat{\mathbf{w}}_{MAP}\|$	$\|\hat{\mathbf{w}}_{MAP} - \hat{\mathbf{w}}_{ML}\|$
10^{-10}	72.07	0.0000	0.549
10^{-8}	52.72	0.0015	0.549
10^{-6}	4.719	0.0089	0.549
7.9×10^{-6} (best)	**4.427**	0.0094	0.549
10^{-4}	4.504	0.0120	0.548
10^{-2}	4.703	0.107	0.468
10^{0}	5.294	0.501	0.069
10^{2}	5.453	0.549	0.0009
$10^{4}, 10^{6}$	5.455	0.549	0.0000

Table 3: MAP results versus γ (ML: validation MSE 5.455, $\|\hat{\mathbf{w}}_{ML}\| = 0.549$).

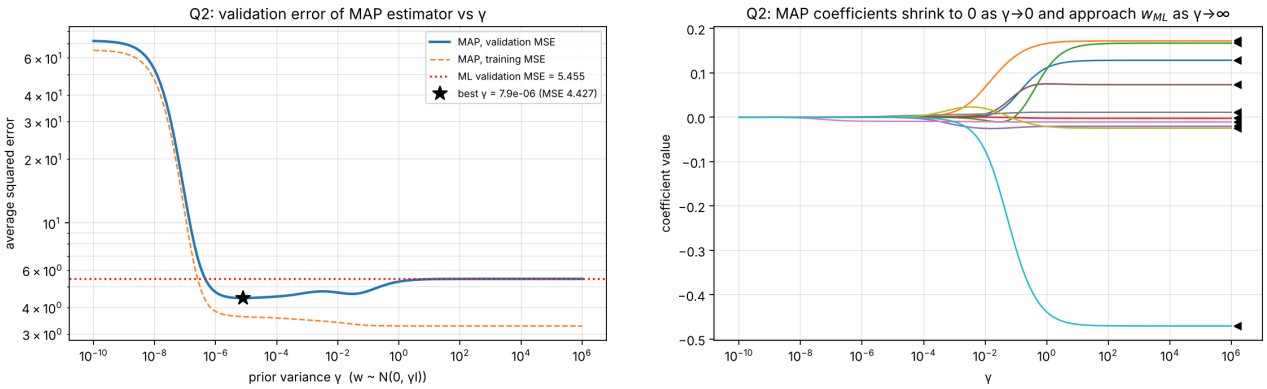


Figure 6: Left: training/validation MSE of the MAP estimator vs. γ with the ML level (red). Right: MAP coefficients vs. γ; black triangles mark $\hat{\mathbf{w}}_{ML}$.

To check that the trend is not specific to one random draw, the whole experiment was repeated on 100 independent train/validation draws:

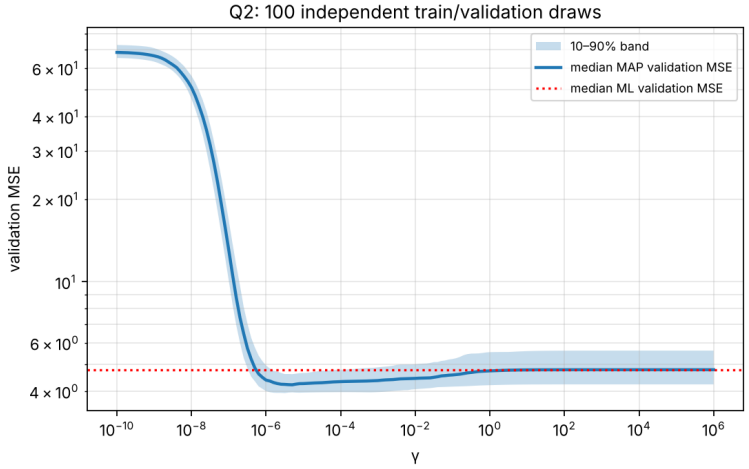


Figure 7: Median and 10–90% band of MAP validation MSE over 100 draws. Median ML validation MSE 4.79; median best-MAP 4.20; best γ median 4×10^{-6} (IQR 2×10^{-6}–10^{-5}). The best MAP model beat ML in 100/100 draws.

Discussion

- **U-shaped behavior in γ.** For very small γ the prior forces $\mathbf{w} \approx \mathbf{0}$, the model predicts $y \approx 0$ everywhere and validation MSE (≈ 72) is roughly the variance of y – severe underfitting. Increasing γ lets the data speak; validation error reaches a minimum of 4.43 at $\gamma \approx 8 \times 10^{-6}$, then rises and plateaus at the ML value (5.455) once $\gamma \gtrsim 10^2$, where $\hat{\mathbf{w}}_{MAP}$ and $\hat{\mathbf{w}}_{ML}$ coincide to four decimals.
- **MAP vs. ML.** ML is the $\gamma \to \infty$ limit of MAP. ML has the lowest training error (3.27) but a large train/validation gap (3.27 vs. 5.46): with 100 samples, 10 parameters, and data that are not truly cubic-plus-Gaussian (they come from a 3-component Gaussian mixture), ML overfits. A

well-chosen prior trades a small increase in training error (3.62) for a 19% reduction in validation error.

- **Why the best γ is so small.** The features are unscaled: x_1 ranges roughly over $[-15, 15]$, so cubic features are $\sim 10^3$ while the constant and linear features are ~ 1. The isotropic prior γI therefore penalizes low-order coefficients far more, in relative terms, than high-order ones. At the optimum essentially only the large-magnitude cubic terms survive ($\hat{w}_{x_1^3} = -0.0093$, which captures the dominant trend $y \approx -0.0093\, x_1^3$ through the three mixture components at $x_1 \in \{-10, 0, 10\}$). The prior thus acts as automatic model simplification. Standardizing the features first would make the prior act more evenly and move the optimal γ toward $O(1)$.
- In practice γ (or λ) would be selected by cross-validation on training data only; here the validation set is used to visualize the bias–variance tradeoff, as the question asks.

Question 3

MAP objective

Let $\mathbf{p} = [x, y]^T$, $d_i(\mathbf{p}) = \|\mathbf{p} - [x_i, y_i]^T\|$, $\mathbf{r} = [r_1, \ldots, r_K]^T$. With independent noise,

$$p(\mathbf{r}|\mathbf{p}) = \prod_{i=1}^{K} \frac{1}{\sqrt{2\pi}\sigma_i} \exp\Big(- \frac{(r_i - d_i(\mathbf{p}))^2}{2\sigma_i^2}\Big), \qquad p(\mathbf{p}) = \frac{1}{2\pi\sigma_x\sigma_y} \exp\Big(- \frac{1}{2}\Big[\frac{x^2}{\sigma_x^2} + \frac{y^2}{\sigma_y^2}\Big]\Big).$$

$$\hat{\mathbf{p}}_{MAP} = \arg\max_{\mathbf{p}}\, p(\mathbf{p}|\mathbf{r}) = \arg\max_{\mathbf{p}}\, p(\mathbf{r}|\mathbf{p})\, p(\mathbf{p}) = \arg\max_{\mathbf{p}} \Big[\ln p(\mathbf{r}|\mathbf{p}) + \ln p(\mathbf{p})\Big].$$

Dropping the evidence $p(\mathbf{r})$ and all normalization constants (independent of $\mathbf{p}$), taking logs (monotone), multiplying by -2 (turning max into min):

$$\boxed{[x_{MAP}, y_{MAP}]^T = \arg\min_{x,y}\; J(x,y) = \sum_{i=1}^{K} \frac{\Big(r_i - \sqrt{(x-x_i)^2 + (y-y_i)^2}\Big)^2}{\sigma_i^2} + \frac{x^2}{\sigma_x^2} + \frac{y^2}{\sigma_y^2}}$$

(The rejection of negative ranges makes the true likelihood a truncated Gaussian; following the question, this is ignored in the objective.) With $\sigma_i = \sigma = 0.3$ for all i, one could further multiply by σ^2, but keeping J as above makes its values comparable across K.

Implementation

The code (Appendix, `q3.py`) (1) draws the true position uniformly inside the unit circle, $\mathbf{p}_T = (-0.415, -0.397)$; (2) for each $K \in \{1, 2, 3, 4\}$ places landmarks at angles $2\pi i/K$, $i = 0, \ldots, K-1$ on the unit circle; (3) draws $r_i = d_i(\mathbf{p}_T) + n_i$, $n_i \sim \mathcal{N}(0, 0.3^2)$, resampling any negative r_i; (4) evaluates J on an 801×801 grid over $[-2, 2]^2$ with $\sigma_x = \sigma_y = 0.25$; (5) draws contours at the *same* levels $\{0.5, 1, 2, 4, 8, 16, 32, 64, 128\}$ for every K, marks the true position (+), landmarks (o), and the grid minimizer ($\times$); and (6) shows the region $J \le J_{\min} + 2$ (red dashed), an approximate 1-σ-like uncertainty region of the posterior, whose area is reported.

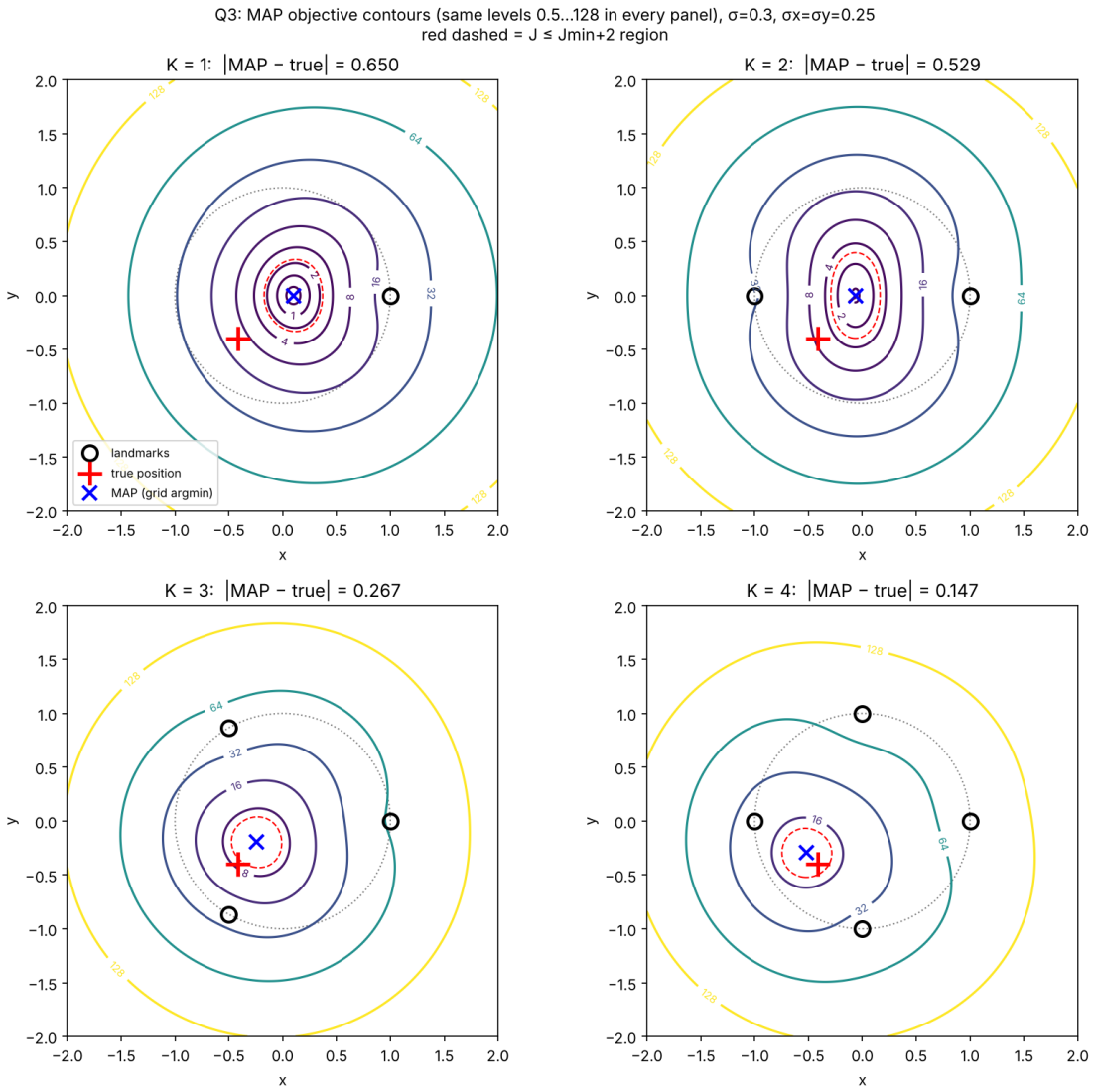


Figure 8: MAP objective contours for $K = 1, \ldots, 4$ at identical levels.

K	$\hat{\mathbf{p}}_{MAP}$	$\|\hat{\mathbf{p}}_{MAP} - \mathbf{p}_T\|$	area of $\{J \le J_{\min} + 2\}$	Monte Carlo mean error / area (500 runs)
1	(0.100, 0.000)	0.650	0.287	0.558 / 0.311
2	(−0.065, 0.000)	0.529	0.292	0.515 / 0.286
3	(−0.240, −0.195)	0.267	0.173	0.374 / 0.201
4	(−0.520, −0.295)	0.147	0.165	0.332 / 0.172

Table 4: MAP estimates for the plotted realization, plus averages over 500 random true positions and noise draws.

Discussion

- $K = 1$**:** one range only constrains the vehicle to a ring of radius r_1 around the landmark; it says nothing about direction. The prior pulls the estimate to the point of that ring nearest the origin,

so the MAP lies on the line joining the landmark and the origin, at $(0.10, 0)$, far from the truth. Contours are nearly circular and centered near the origin: the prior dominates.

- $K = 2$**:** the two landmarks $(\pm 1, 0)$ are collinear with the origin, so the problem is symmetric under $y \to -y$: the two range rings intersect at two mirror points and the data cannot tell them apart. The prior resolves this by placing the MAP on the axis ($y = 0$), and the contours are elongated vertically (high uncertainty in y, good information in x).
- $K = 3, 4$**:** three or more non-collinear landmarks triangulate the position uniquely. The innermost contours move off the origin toward the true position (error 0.27, then 0.15), and the bowl becomes visibly steeper: at the same contour levels, the curves are more tightly packed, and the low levels (0.5–8) disappear because more measurement terms add to J.
- **Closer and more certain as K grows.** Yes, both in this realization and on average: the Monte Carlo mean error falls from 0.56 to 0.33 and the area of the $J \le J_{\min} + 2$ region shrinks from 0.31 to 0.17. More measurements make the likelihood sharper relative to the fixed prior, so the posterior concentrates near the truth. Some residual pull toward the origin remains for all K because the prior ($\sigma_x = \sigma_y = 0.25$) is strong compared with the range noise ($\sigma = 0.3$); a weaker prior would let the estimate move further toward the data.

Question 4 (Duda–Hart–Stork Problem 2.13)

The conditional risk of action α_i is $R(\alpha_i|\mathbf{x}) = \sum_{j=1}^{c} \lambda(\alpha_i|\omega_j) P(\omega_j|\mathbf{x})$.

Deciding a class α_i, $i \in \{1, \ldots, c\}$: the loss is 0 when $j = i$ and λ_s otherwise, so

$$R(\alpha_i|\mathbf{x}) = \lambda_s \sum_{j \neq i} P(\omega_j|\mathbf{x}) = \lambda_s \big[1 - P(\omega_i|\mathbf{x})\big].$$

Rejecting (α_{c+1}): the loss is λ_r for every true class, so

$$R(\alpha_{c+1}|\mathbf{x}) = \lambda_r \sum_{j=1}^{c} P(\omega_j|\mathbf{x}) = \lambda_r.$$

The Bayes (minimum-risk) rule picks the action with the smallest conditional risk.

1. Among the c classification actions, $\lambda_s[1 - P(\omega_i|\mathbf{x})]$ is smallest for the class with the largest posterior, so if we classify at all we choose ω_i with $P(\omega_i|\mathbf{x}) \ge P(\omega_j|\mathbf{x})$ for all j.
2. This best classification beats rejection iff

$$\lambda_s \big[1 - P(\omega_i|\mathbf{x})\big] \le \lambda_r \iff P(\omega_i|\mathbf{x}) \ge 1 - \frac{\lambda_r}{\lambda_s}.$$

Hence the minimum risk is obtained by deciding ω_i if $P(\omega_i|\mathbf{x}) \ge P(\omega_j|\mathbf{x})\ \forall j$ *and* $P(\omega_i|\mathbf{x}) \ge 1 - \lambda_r/\lambda_s$, and rejecting otherwise. Since the overall risk $R = \int R(\alpha(\mathbf{x})|\mathbf{x}) p(\mathbf{x})\, d\mathbf{x}$ is minimized by minimizing the conditional risk at every $\mathbf{x}$, this rule minimizes the total risk. ■

If $\lambda_r = 0$**:** the threshold is $1 - 0 = 1$, so we classify only when $P(\omega_i|\mathbf{x}) = 1$ (complete certainty) and otherwise reject. Rejection is free, so we always reject whenever there is any chance of error.

If $\lambda_r > \lambda_s$**:** the threshold $1 - \lambda_r/\lambda_s < 0$ is satisfied by every posterior, so we never reject; the rule reduces to the ordinary MAP (minimum-error) classifier. Rejecting costs more than even a wrong decision. (More generally, since $\max_i P(\omega_i|\mathbf{x}) \ge 1/c$, rejection never occurs once $\lambda_r \ge \frac{c-1}{c}\lambda_s$.)

Question 5

With 1-of-K coding, $P(\mathbf{z}|\Theta) = \prod_{k=1}^{K} \theta_k^{z_k}$, with $\theta_k \geq 0$, $\sum_k \theta_k = 1$. Let $N_k = \sum_{n=1}^{N} z_{nk}$ be the number of samples in state k ($\sum_k N_k = N$).

ML estimator

$$p(D|\Theta) = \prod_{n=1}^{N} \prod_{k=1}^{K} \theta_k^{z_{nk}} = \prod_{k=1}^{K} \theta_k^{N_k}, \qquad \ln p(D|\Theta) = \sum_{k=1}^{K} N_k \ln \theta_k.$$

Maximize subject to $\sum_k \theta_k = 1$ with a Lagrange multiplier:

$$\mathcal{L} = \sum_k N_k \ln \theta_k + \lambda\Big(1 - \sum_k \theta_k\Big), \qquad \frac{\partial \mathcal{L}}{\partial \theta_k} = \frac{N_k}{\theta_k} - \lambda = 0 \Rightarrow \theta_k = \frac{N_k}{\lambda}.$$

Summing over k: $1 = \sum_k N_k/\lambda = N/\lambda \Rightarrow \lambda = N$. Thus

$$\boxed{\hat{\theta}_k^{ML} = \frac{N_k}{N} = \frac{1}{N}\sum_{n=1}^{N} z_{nk}, \qquad \hat{\Theta}_{ML} = \frac{1}{N}\sum_{n=1}^{N} \mathbf{z}_n,}$$

the relative frequency of each state. It is the global maximum because $\sum_k N_k \ln \theta_k$ is concave and the constraint set is convex.

MAP estimator with Dirichlet prior

$$p(\Theta|D) \propto p(D|\Theta)p(\Theta|\alpha) \propto \prod_{k=1}^{K} \theta_k^{N_k} \cdot \prod_{k=1}^{K} \theta_k^{\alpha_k - 1} = \prod_{k=1}^{K} \theta_k^{N_k + \alpha_k - 1}$$

(the posterior is again Dirichlet, with parameters $N_k + \alpha_k$; $B(\alpha)$ does not depend on Θ and is dropped). Maximizing $\sum_k (N_k + \alpha_k - 1) \ln \theta_k$ subject to $\sum_k \theta_k = 1$ exactly as above gives $\theta_k = (N_k + \alpha_k - 1)/\lambda$ with $\lambda = \sum_k (N_k + \alpha_k - 1) = N + \sum_k \alpha_k - K$:

$$\boxed{\hat{\theta}_k^{MAP} = \frac{N_k + \alpha_k - 1}{N + \sum_{j=1}^{K} \alpha_j - K}.}$$

This holds when $N_k + \alpha_k > 1$ for all k (e.g. all $\alpha_k \geq 1$), so that the objective is concave with an interior maximum; if some $N_k + \alpha_k < 1$ the posterior density is unbounded at $\theta_k \to 0$ and the mode lies on the boundary.

Interpretation. $\alpha_k - 1$ acts as a pseudo-count added to the observed count of state k. With a uniform prior ($\alpha_k = 1$) the MAP equals the ML estimate; $\alpha_k = 2$ gives Laplace (add-one) smoothing, which avoids zero probabilities for unseen states; and as $N \to \infty$ the data dominate and $\hat{\Theta}_{MAP} \to \hat{\Theta}_{ML}$.

Acknowledgement of resources and AI use

[Review and edit this statement so that it accurately describes how you used these tools before submitting.] I used Claude (Anthropic, model Claude Opus 5.5, via claude.ai, October 25, 2026) as a coding and writing assistant for this assignment. It helped write the Python implementations

in the Appendix, run the experiments, produce the figures, draft the derivations for Questions 1–5, and typeset this report. I reviewed the derivations and results. The data generator for Question 2 is the course-provided `hw2q2.py`. Libraries used: NumPy, SciPy (`scipy.stats.multivariate_normal`, `scipy.optimize.minimize`), Matplotlib. Question 4 is Problem 2.13 from R. O. Duda, P. E. Hart, D. G. Stork, *Pattern Classification*, 2nd ed., Wiley, 2001.

Appendix: Code

q1.py

```
"""EECE5644 Assignment 2 - Question 1
Min-P(error) classifier for a 2-class Gaussian-mixture problem, plus ML-trained
logistic-linear and logistic-quadratic approximations of the class posterior.
"""
import numpy as np
from scipy.stats import multivariate_normal as mvn
from scipy.optimize import minimize
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import os

rng = np.random.default_rng(5644)
os.makedirs("figs", exist_ok=True)

# ---------------- True data distribution ----------------
priors = np.array([0.6, 0.4])
means = {0: [np.array([-0.9, -1.1]), np.array([0.8, 0.75])],
         1: [np.array([-1.1, 0.9]), np.array([0.9, -0.75])]}
C = np.diag([0.75, 1.25])
w_mix = [0.5, 0.5]


def generate(N):
    labels = (rng.random(N) >= priors[0]).astype(int)          # P(L=1)=0.4
    X = np.zeros((N, 2))
    for l in (0, 1):
        idx = np.where(labels == l)[0]
        comp = (rng.random(idx.size) >= w_mix[0]).astype(int)
        for c in (0, 1):
            sub = idx[comp == c]
            X[sub] = rng.multivariate_normal(means[l][c], C, size=sub.size)
    return X, labels


def class_pdf(X, l):
    return sum(w_mix[c] * mvn.pdf(X, means[l][c], C) for c in (0, 1))


datasets = {n: generate(n) for n in (20, 200, 2000)}
Xv, Lv = generate(10000)

# ---------------- Part 1: theoretically optimal classifier ----------------
# Decide 1 iff ln p(x|1) - ln p(x|0) > ln(P0/P1)   (0-1 loss -> MAP rule)
score = np.log(class_pdf(Xv, 1)) - np.log(class_pdf(Xv, 0))
gamma_map = np.log(priors[0] / priors[1])


def roc(score, labels):
    thr = np.concatenate(([-np.inf], np.sort(score), [np.inf]))
    n1, n0 = (labels == 1).sum(), (labels == 0).sum()
    order = np.argsort(score)
    s_sorted, l_sorted = score[order], labels[order]
    # decisions: 1 if score > thr ; count of positives above each threshold
    tp = np.concatenate(([n1], n1 - np.cumsum(l_sorted == 1), [0]))
    fp = np.concatenate(([n0], n0 - np.cumsum(l_sorted == 0), [0]))
    return fp / n0, tp / n1, thr


def perr(dec, labels):
    return np.mean(dec != labels)


pfa, pd, thr = roc(score, Lv)
```

```
p_err_emp = (pfa * (Lv == 0).mean() + (1 - pd) * (Lv == 1).mean())
i_emp = np.argmin(p_err_emp)
dec_opt = (score > gamma_map).astype(int)
pe_opt = perr(dec_opt, Lv)
pfa_opt = np.mean(dec_opt[Lv == 0] == 1)
pd_opt = np.mean(dec_opt[Lv == 1] == 1)

print(f"Validation class counts: N0={np.sum(Lv==0)}, N1={np.sum(Lv==1)}")
print(f"Theoretical threshold gamma = P0/P1 = {priors[0]/priors[1]:.3f} (ln = {gamma_map:.4f})")
print(f"Min-P(error) classifier on D10K: P(error) = {pe_opt:.4f}  (P_FA={pfa_opt:.4f}, P_D={pd_opt:.4f})")
print(f"Empirically best threshold on ROC: ln gamma = {thr[i_emp]:.4f}, P(error) = {p_err_emp[i_emp]:.4f}")
counts = {(d, l): int(np.sum((dec_opt == d) & (Lv == l))) for d in (0, 1) for l in (0, 1)}
print("Decision/label counts (D,L):", counts)

fig, ax = plt.subplots(figsize=(5.5, 5))
ax.plot(pfa, pd, lw=1.8, label="ROC (log-likelihood-ratio score)")
ax.plot(pfa_opt, pd_opt, "r*", ms=15, label=f"Theoretical min-P(error), γ=1.5\nP(error)={pe_opt:.4f}")
ax.plot(pfa[i_emp], pd[i_emp], "go", mfc="none", ms=11, label=f"Empirical min-P(error)\nP(error)={p_err_emp[i_emp]:.4f}")
ax.plot([0, 1], [0, 1], "k:", lw=0.8)
ax.set_xlabel("P(false alarm)  P(D=1|L=0)")
ax.set_ylabel("P(correct detection)  P(D=1|L=1)")
ax.set_title("Q1 Part 1: ROC of the optimal classifier on $D^{10K}_{validate}$")
ax.legend(loc="lower right", fontsize=8)
ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig("figs/q1_roc.png", dpi=160); plt.close(fig)

# grid for decision boundaries
gx, gy = np.meshgrid(np.linspace(-4.5, 4.5, 300), np.linspace(-5, 5, 300))
G = np.column_stack([gx.ravel(), gy.ravel()])
g_score = (np.log(class_pdf(G, 1)) - np.log(class_pdf(G, 0)) - gamma_map).reshape(gx.shape)


def scatter_with_boundary(ax, X, L, dec, Z, title):
    correct = dec == L
    for l, mk in ((0, "o"), (1, "^")):
        ax.scatter(*X[(L == l) & correct].T, s=6, marker=mk, c="tab:green", alpha=0.5,
                   label=f"L={l} correct")
        ax.scatter(*X[(L == l) & ~correct].T, s=8, marker=mk, c="tab:red", alpha=0.7,
                   label=f"L={l} incorrect")
    ax.contour(gx, gy, Z, levels=[0], colors="k", linewidths=1.6)
    ax.set_title(title, fontsize=9); ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$")
    ax.set_xlim(-4.5, 4.5); ax.set_ylim(-5, 5)


fig, ax = plt.subplots(figsize=(6, 5.5))
scatter_with_boundary(ax, Xv, Lv, dec_opt, g_score,
                      f"Optimal (MAP) decision boundary on $D^{{10K}}_{{validate}}$, P(error)={pe_opt:.4f}")
ax.legend(fontsize=7, markerscale=2, loc="upper right")
fig.tight_layout(); fig.savefig("figs/q1_opt_boundary.png", dpi=160); plt.close(fig)

# ---------------- Part 2: logistic models trained by ML ----------------
def z_lin(X):
    return np.column_stack([np.ones(len(X)), X])


def z_quad(X):
    x1, x2 = X[:, 0], X[:, 1]
    return np.column_stack([np.ones(len(X)), x1, x2, x1**2, x1 * x2, x2**2])


def nll(w, Z, L):
    a = Z @ w
    # -sum[ l*log h + (1-l)*log(1-h) ] written stably with logaddexp
    return np.mean(np.logaddexp(0, a) - L * a)


def nll_grad(w, Z, L):
    h = 1 / (1 + np.exp(-(Z @ w)))
    return Z.T @ (h - L) / len(L)
```

```
def train(Z, L):
    res = minimize(nll, np.zeros(Z.shape[1]), args=(Z, L), jac=nll_grad,
                   method="BFGS", options={"gtol": 1e-6, "maxiter": 5000})
    return res.x, res


results = {}
for name, zf in (("linear", z_lin), ("quadratic", z_quad)):
    fig, axes = plt.subplots(2, 3, figsize=(15, 9.5))
    for j, n in enumerate((20, 200, 2000)):
        Xt, Lt = datasets[n]
        w, res = train(zf(Xt), Lt)
        # classify: h(x)≈P(L=1|x); min-P(error) rule decides 1 iff h>=0.5 (i.e. w^T z >= 0)
        dec_t = (zf(Xt) @ w >= 0).astype(int)
        dec_v = (zf(Xv) @ w >= 0).astype(int)
        pe_t, pe_v = perr(dec_t, Lt), perr(dec_v, Lv)
        results[(name, n)] = dict(w=w, pe_train=pe_t, pe_val=pe_v, nll=res.fun, it=res.nit, ok=res.success)
        print(f"logistic-{name:9s} N={n:5d}: train P(err)={pe_t:.4f}  validate P(err)={pe_v:.4f}  "
              f"NLL/N={res.fun:.4f} iters={res.nit} converged={res.success}  w={np.round(w,3)}")
        Zg = (zf(G) @ w).reshape(gx.shape)
        scatter_with_boundary(axes[0, j], Xt, Lt, dec_t, Zg,
                              f"logistic-{name}, trained on $D^{{{n}}}_{{train}}$ (train P(err)={pe_t:.3f})")
        axes[0, j].contour(gx, gy, g_score, levels=[0], colors="gray", linestyles="--", linewidths=1)
        scatter_with_boundary(axes[1, j], Xv, Lv, dec_v, Zg,
                              f"same model on $D^{{10K}}_{{validate}}$ (P(err)={pe_v:.4f})")
        axes[1, j].contour(gx, gy, g_score, levels=[0], colors="gray", linestyles="--", linewidths=1)
    axes[0, 0].legend(fontsize=7, markerscale=2, loc="upper right")
    fig.suptitle(f"Q1 Part 2: logistic-{name} classifiers (black = learned boundary, gray dashed = optimal)")
    fig.tight_layout(); fig.savefig(f"figs/q1_logistic_{name}.png", dpi=140); plt.close(fig)

# summary bar chart
fig, ax = plt.subplots(figsize=(6.5, 4))
xs = np.arange(3)
for k, name in enumerate(("linear", "quadratic")):
    vals = [results[(name, n)]["pe_val"] for n in (20, 200, 2000)]
    ax.bar(xs + (k - 0.5) * 0.35, vals, 0.35, label=f"logistic-{name}")
    for x, v in zip(xs + (k - 0.5) * 0.35, vals):
        ax.text(x, v + 0.005, f"{v:.3f}", ha="center", fontsize=8)
ax.axhline(pe_opt, color="r", ls="--", label=f"optimal classifier ({pe_opt:.3f})")
ax.set_xticks(xs, ["$N=20$", "$N=200$", "$N=2000$"])
ax.set_ylabel("Estimated P(error) on $D^{10K}_{validate}$")
ax.set_title("Q1: validation P(error) vs training-set size and model form")
ax.legend(fontsize=8); ax.set_ylim(0, 0.6)
fig.tight_layout(); fig.savefig("figs/q1_summary.png", dpi=160); plt.close(fig)

np.save("figs/q1_results.npy", {"pe_opt": pe_opt, "results": results}, allow_pickle=True)
```

q2.py

```
"""EECE5644 Assignment 2 - Question 2
ML and MAP estimation of a cubic polynomial regression y = c(x,w) + v, x in R^2.
Data generator reproduced from the course-provided hw2q2.py (plotting removed,
seeded RNG added for reproducibility). Ntrain=100, Nvalidate=1000.
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import os

os.makedirs("figs", exist_ok=True)
np.random.seed(5644)


# ---------- course-provided generator (hw2q2.py), unchanged logic ----------
```

```
def generateData(N):
    priors = [.3, .4, .3]
    meanVectors = np.array([[-10, 0, 10], [0, 0, 0], [10, 0, -10]])
    cov = np.zeros((3, 3, 3))
    cov[:, :, 0] = [[1, 0, -3], [0, 1, 0], [-3, 0, 15]]
    cov[:, :, 1] = [[8, 0, 0], [0, .5, 0], [0, 0, .5]]
    cov[:, :, 2] = [[1, 0, -3], [0, 1, 0], [-3, 0, 15]]
    x = np.zeros((3, N))
    u = np.random.random(N)
    thresholds = np.append(np.cumsum(priors), 1)
    for l in range(3):
        ind = np.where(u <= thresholds[l])[0]
        u[ind] = 1.1
        x[:, ind] = np.random.multivariate_normal(meanVectors[:, l], cov[:, :, l], len(ind)).T
    return x


def hw2q2(Ntrain=100, Nvalidate=1000):
    d = generateData(Ntrain); xT, yT = d[0:2, :].T, d[2, :]
    d = generateData(Nvalidate); xV, yV = d[0:2, :].T, d[2, :]
    return xT, yT, xV, yV


# ---------- model ----------
def z_cubic(X):
    """All monomials of total degree <= 3 in (x1,x2): 10 features."""
    x1, x2 = X[:, 0], X[:, 1]
    return np.column_stack([np.ones(len(X)), x1, x2,
                            x1**2, x1 * x2, x2**2,
                            x1**3, x1**2 * x2, x1 * x2**2, x2**3])


def w_ml(Z, y):
    return np.linalg.solve(Z.T @ Z, Z.T @ y)


def w_map(Z, y, gamma, sigma2):
    return np.linalg.solve(Z.T @ Z + (sigma2 / gamma) * np.eye(Z.shape[1]), Z.T @ y)


def mse(Z, y, w):
    return np.mean((y - Z @ w) ** 2)


gammas = np.logspace(-10, 6, 161)

# ---------- main experiment (one dataset) ----------
xT, yT, xV, yV = hw2q2()
ZT, ZV = z_cubic(xT), z_cubic(xV)
wML = w_ml(ZT, yT)
resid = yT - ZT @ wML
sigma2 = resid @ resid / (len(yT) - ZT.shape[1])   # unbiased noise-variance estimate from ML fit
mse_ml_train, mse_ml_val = mse(ZT, yT, wML), mse(ZV, yV, wML)
print(f"sigma^2 estimate (from ML residuals) = {sigma2:.4f}")
print(f"ML : train MSE = {mse_ml_train:.4f}, validation MSE = {mse_ml_val:.4f}")
print("w_ML =", np.round(wML, 4))

W = np.array([w_map(ZT, yT, g, sigma2) for g in gammas])
mse_map_val = np.array([mse(ZV, yV, w) for w in W])
mse_map_tr = np.array([mse(ZT, yT, w) for w in W])
ib = np.argmin(mse_map_val)
print(f"MAP best gamma = {gammas[ib]:.3g}: validation MSE = {mse_map_val[ib]:.4f} "
      f"(train MSE {mse_map_tr[ib]:.4f})")
for g in (1e-10, 1e-8, 1e-6, 1e-4, 1e-2, 1e0, 1e2, 1e4, 1e6):
    w = w_map(ZT, yT, g, sigma2)
    print(f"  gamma=1e{int(np.log10(g)):+d}: val MSE={mse(ZV, yV, w):9.4f}  ||w||={np.linalg.norm(w):.4f}  "
          f"||w_MAP - w_ML||={np.linalg.norm(w - wML):.4f}")
print("w_MAP(best) =", np.round(W[ib], 4))
```

```
# ---------- repeated trials to show the trend isn't a one-dataset artifact ----------
R = 100
curves = np.zeros((R, len(gammas))); ml_vals = np.zeros(R)
for r in range(R):
    a, b, c, d = hw2q2()
    Za, Zc = z_cubic(a), z_cubic(c)
    wm = w_ml(Za, b)
    s2 = np.sum((b - Za @ wm) ** 2) / (len(b) - 10)
    ml_vals[r] = mse(Zc, d, wm)
    curves[r] = [mse(Zc, d, w_map(Za, b, g, s2)) for g in gammas]
best_g = gammas[np.argmin(curves, axis=1)]
print(f"\n{R} trials: median ML val MSE = {np.median(ml_vals):.3f}; "
      f"median best-MAP val MSE = {np.median(curves.min(axis=1)):.3f}; "
      f"median best gamma = {np.median(best_g):.3g} (IQR {np.percentile(best_g,25):.3g}-{np.percentile(best_g,75):.3g})")
print(f"fraction of trials where best MAP beats ML: {np.mean(curves.min(axis=1) < ml_vals):.2f}")

# ---------- figures ----------
fig = plt.figure(figsize=(11, 4.5))
for k, (X, y, t) in enumerate(((xT, yT, "Training set (N=100)"), (xV, yV, "Validation set (N=1000)"))):
    ax = fig.add_subplot(1, 2, k + 1, projection="3d")
    ax.scatter(X[:, 0], X[:, 1], y, s=6)
    ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$"); ax.set_zlabel("y"); ax.set_title(t)
fig.tight_layout(); fig.savefig("figs/q2_data.png", dpi=140); plt.close(fig)

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.semilogx(gammas, mse_map_val, lw=2, label="MAP, validation MSE")
ax.semilogx(gammas, mse_map_tr, lw=1.2, ls="--", label="MAP, training MSE")
ax.axhline(mse_ml_val, color="r", ls=":", label=f"ML validation MSE = {mse_ml_val:.3f}")
ax.plot(gammas[ib], mse_map_val[ib], "k*", ms=12, label=f"best γ = {gammas[ib]:.2g} (MSE {mse_map_val[ib]:.3f})")
ax.set_xlabel("prior variance γ  (w ~ N(0, γI))"); ax.set_ylabel("average squared error")
ax.set_title("Q2: validation error of MAP estimator vs γ")
ax.set_yscale("log"); ax.legend(fontsize=8); ax.grid(alpha=0.3, which="both")
fig.tight_layout(); fig.savefig("figs/q2_mse_vs_gamma.png", dpi=160); plt.close(fig)

fig, ax = plt.subplots(figsize=(7, 4.5))
for j in range(10):
    ax.semilogx(gammas, W[:, j], lw=1.2)
ax.set_xlabel("γ"); ax.set_ylabel("coefficient value")
ax.set_title("Q2: MAP coefficients shrink to 0 as γ→0 and approach $w_{ML}$ as γ→∞")
for j in range(10):
    ax.plot(gammas[-1] * 1.6, wML[j], "k<", ms=5)
ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig("figs/q2_coeff_path.png", dpi=160); plt.close(fig)

fig, ax = plt.subplots(figsize=(7, 4.5))
lo, med, hi = np.percentile(curves, [10, 50, 90], axis=0)
ax.fill_between(gammas, lo, hi, alpha=0.25, label="10–90% band")
ax.semilogx(gammas, med, lw=2, label="median MAP validation MSE")
ax.axhline(np.median(ml_vals), color="r", ls=":", label="median ML validation MSE")
ax.set_xscale("log"); ax.set_yscale("log")
ax.set_xlabel("γ"); ax.set_ylabel("validation MSE"); ax.legend(fontsize=8); ax.grid(alpha=0.3, which="both")
ax.set_title(f"Q2: {R} independent train/validation draws")
fig.tight_layout(); fig.savefig("figs/q2_mse_trials.png", dpi=160); plt.close(fig)
```

q3.py

```
"""EECE5644 Assignment 2 - Question 3
MAP vehicle localization from K noisy range measurements to landmarks on the unit circle.
Objective (to minimize):  J(x,y) = sum_i (r_i - d_i(x,y))^2 / sigma_i^2 + x^2/sx^2 + y^2/sy^2
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import os

os.makedirs("figs", exist_ok=True)
```

```
rng = np.random.default_rng(5644)

sigma = 0.3          # range-noise std (all landmarks)
sx = sy = 0.25       # prior std about the origin

# true position: uniform inside the unit circle
rad, ang = np.sqrt(rng.random()), 2 * np.pi * rng.random()
pT = np.array([rad * np.cos(ang), rad * np.sin(ang)])
print(f"True position = ({pT[0]:.4f}, {pT[1]:.4f})")


def landmarks(K):
    th = 2 * np.pi * np.arange(K) / K
    return np.column_stack([np.cos(th), np.sin(th)])


def measure(L, p):
    d = np.linalg.norm(L - p, axis=1)
    r = d + sigma * rng.standard_normal(len(d))
    while np.any(r < 0):                        # reject and resample negative ranges
        bad = r < 0
        r[bad] = d[bad] + sigma * rng.standard_normal(bad.sum())
    return r


def J(X, Y, L, r):
    val = X**2 / sx**2 + Y**2 / sy**2
    for (lx, ly), ri in zip(L, r):
        val = val + (ri - np.hypot(X - lx, Y - ly)) ** 2 / sigma**2
    return val


g = np.linspace(-2, 2, 801)
X, Y = np.meshgrid(g, g)
levels = [0.5, 1, 2, 4, 8, 16, 32, 64, 128]   # identical contour levels for every K

fig, axes = plt.subplots(2, 2, figsize=(11, 10.5))
for K, ax in zip((1, 2, 3, 4), axes.ravel()):
    L = landmarks(K)
    r = measure(L, pT)
    Z = J(X, Y, L, r)
    i = np.unravel_index(np.argmin(Z), Z.shape)
    pMAP = np.array([X[i], Y[i]])
    # area enclosed by J <= Jmin + 2 (an approximate "uncertainty region")
    area = np.sum(Z <= Z.min() + 2) * (g[1] - g[0]) ** 2
    print(f"K={K}: ranges={np.round(r,3)}  MAP≈({pMAP[0]:.3f},{pMAP[1]:.3f})  "
          f"|MAP-true|={np.linalg.norm(pMAP-pT):.3f}  Jmin={Z.min():.3f}  area(J<=Jmin+2)={area:.4f}")
    cs = ax.contour(X, Y, Z, levels=levels, cmap="viridis")
    ax.clabel(cs, fontsize=7, fmt="%g")
    ax.contour(X, Y, Z, levels=[Z.min() + 2], colors="r", linewidths=1, linestyles="--")
    ax.plot(*L.T, "o", mfc="none", mec="k", ms=10, mew=2, label="landmarks")
    ax.plot(*pT, "+", c="r", ms=16, mew=2.5, label="true position")
    ax.plot(*pMAP, "x", c="b", ms=10, mew=2, label="MAP (grid argmin)")
    ax.add_patch(plt.Circle((0, 0), 1, fill=False, ls=":", color="gray"))
    ax.set_title(f"K = {K}:  |MAP − true| = {np.linalg.norm(pMAP-pT):.3f}")
    ax.set_xlim(-2, 2); ax.set_ylim(-2, 2); ax.set_aspect("equal")
    ax.set_xlabel("x"); ax.set_ylabel("y")
axes[0, 0].legend(fontsize=8, loc="lower left")
fig.suptitle(f"Q3: MAP objective contours (same levels {levels[0]}…{levels[-1]} in every panel), "
             f"σ=0.3, σx=σy=0.25\nred dashed = J ≤ Jmin+2 region", fontsize=11)
fig.tight_layout(); fig.savefig("figs/q3_contours.png", dpi=150); plt.close(fig)

# Monte Carlo over many noise draws (and random true positions) to back up the visual trend
T = 500
gc = np.linspace(-1.5, 1.5, 241)
Xc, Yc = np.meshgrid(gc, gc)
err = np.zeros((T, 4)); area = np.zeros((T, 4))
for t in range(T):
    rr, aa = np.sqrt(rng.random()), 2 * np.pi * rng.random()
```

```
    p = np.array([rr * np.cos(aa), rr * np.sin(aa)])
    for k, K in enumerate((1, 2, 3, 4)):
        L = landmarks(K); r = measure(L, p)
        Z = J(Xc, Yc, L, r)
        i = np.unravel_index(np.argmin(Z), Z.shape)
        err[t, k] = np.hypot(Xc[i] - p[0], Yc[i] - p[1])
        area[t, k] = np.sum(Z <= Z.min() + 2) * (gc[1] - gc[0]) ** 2
print(f"\nMonte Carlo ({T} random true positions & noise draws):")
for k in range(4):
    print(f"  K={k+1}: mean |MAP-true| = {err[:,k].mean():.3f}  median = {np.median(err[:,k]):.3f}  "
          f"mean area(J<=Jmin+2) = {area[:,k].mean():.4f}")
```